Team 11

Application Name: Audio Draft

Software Engineering Project

Name (s): Rithwak Somepalli, Christopher Arndall, Brian Smith, Ronald Thorpe, Aymane Rouchdi

Semester: Spring 2026

Group Number: 11

Coordinator: Brian Smith

Name of the Guide: Dr. Tushara Sadasivuni

Date: (02/23/2026)

TABLE OF CONTENTS

1.0..... Planning & Scheduling Table..... 3

2.1.....Revised Problem Statement..... 3

2.2..... New/Improved in Sprint 2..... 3

3.0..... Context Diagram.....4

4.0..... Activity Diagram..... 4

5.1..... Use Cases.....5

5.2..... Requirements..... 11

5.3..... Use Case Diagrams..... 16

6.1..... Database Management..... 18

6.2..... System Database Tables & Relationships.....19

7.0..... Implementation & Screenhsots..... 23

REFERENCES.....

1.0 Planning and Scheduling

Assignee Name	Email address	Task	Duration	Dependency	Due Date	Evaluation
Brian Smith (Coordinator)	bsmith280@student.gsu.edu	Section 1 & 2, Final Formatting, Client Requirements	2	None	02/22/2026	100%, got the client requirements and coordinated everyone
Ronald Thorpe	rthorpe4@student.gsu.edu	Sections 3 & 4	2	Section 2	02/22/2026	100%, did the assigned work
Christopher Arndall	carndall1@student.gsu.edu	Section 5.1 & 5.2 (5 use cases/requirements) + 2 Use Case Diagrams	3	Sections 3-4	02/22/2026	100%, did the assigned work
Aymane Rouchdi	arouchdi1@student.gsu.edu	Section 5.1 & 5.2 (5 use cases/requirements) + 2 Use Case Diagrams	3	Sections 3-4	02/22/2026	100%, did the assigned work
Rithwak Somepalli	rsomepalli1@student.gsu.edu	Sections 6 & 7	3	Section 5	02/23/2026	100%, self

2.1 Revised Problem Statement

Audio draft is a social music intelligence platform designed to support expressive music discovery by transforming natural language descriptions of moods, scenarios, and emotional states into personalized playlists. Merging LLM interpretation with Spotify's audio feature analysis, the system bridges the gap between subjective emotional experiences and data-driven musical matches.

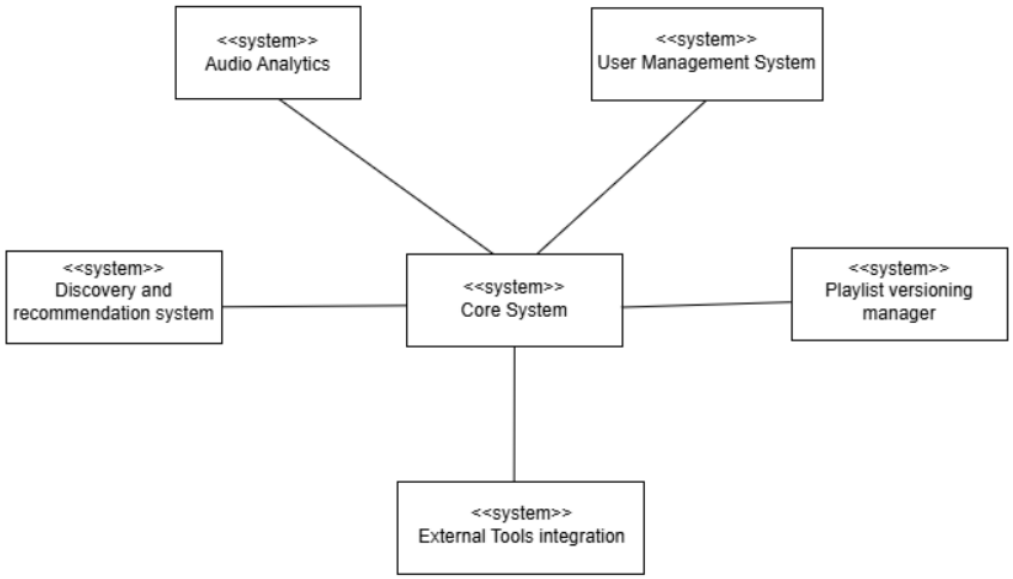
2.2 New/Improved in Sprint 2

Based on client feedback (Group 9) we have enhanced our system with the following features:

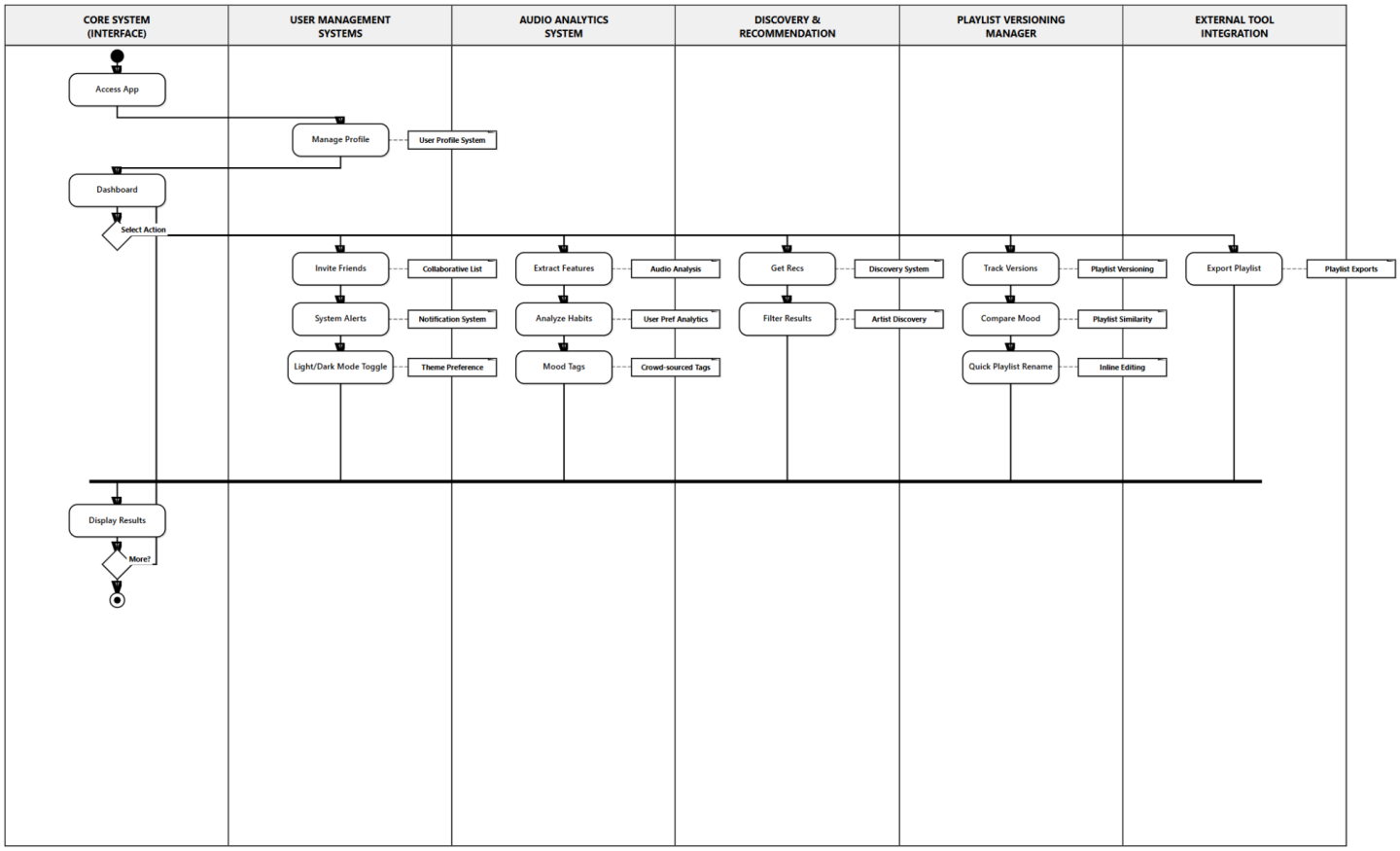
1. Mood Tags - Users can now manually tag songs with emotional descriptors such as "chill", "hyped", "nostalgic", etc. These crowd-sourced tags supplement our LLM analysis, improve recommendation accuracy over time, and create a community-driven emotional map of music
2. Quick Playlist Rename - Streamlined playlist management with inline editing capabilities, reducing the number of clicks required to rename a playlist from multiple menus to a single click. This improves usability and speeds up playlist organization.

3. Light/Dark Mode Toggle - Users can switch between light and dark themes to improve visual comfort and accessibility, especially in different lighting conditions. This feature enhances user experience and aligns with modern application standards.

3.0 Context Diagram



4.0 Activity Diagram



5.1 Use Cases

1. User profile/authentication
2. Create playlist from description
3. Generate playlist from listening history
4. Export playlists
5. Collaborate on playlist
6. Receive notifications
7. Compare playlists
8. Discover artists
9. View trending songs
10. View users music preference analytics
11. Tag songs with mood descriptors
12. Rename playlist
13. Toggle System theme
14. Rate and review songs

Use Case Descriptions

Use Case no.: 1

Use Case Name: User profile / authentication

Actors: User, User Management System

Description:

1. The user navigates to the registration or login page.

2. The user enters required credentials (email and password).

3. The system validates the credentials against stored user data.

4. If valid, the system creates an authenticated session.

5. The user is redirected to the dashboard.

Alternate Path: The user may update profile information (username, theme preference) from the profile settings page.

Exception Path: If credentials are invalid or the email already exists during registration, an error message is displayed.

Pre-condition:

- User must provide valid registration information
- System must have access to the Users database

Post-condition: User is successfully logged in and redirected to the dashboard, or an error message is displayed.

Use Case no.: 2

Use Case Name: Create playlist from description

Actors: User, Core System, Audio Analytics System

Description:

1. The user navigates to “Create Playlist from Description” on the dashboard.
2. The user enters a natural language description of mood or scenario.
3. The system processes the description using the LLM module.
4. The system queries relevant songs using audio features and mood tags.
5. A generated playlist is displayed to the user.
6. The user chooses to save or discard the playlist.

Alternate Path: The user may modify the description and regenerate the playlist.

Exception Path: If the description cannot be processed or no matching songs are found, an error message is displayed.

Pre-condition:

- User must have a valid account
- User must be logged in

Post-condition: A playlist is generated and either saved to the user’s library or discarded.\

Use Case no.: 3

Use Case Name: Generate playlist from user listening history

Actors: User, User Management System, Audio Analytics System

Description:

1. The user selects the “Generate playlist from history” button on the dashboard
2. The system uses the id of the user to grab their listening history
3. Listening history is used by the Audio Analytics System to generate a playlist with tracks related to the user’s history
4. The playlist “Your History Playlist #”is added to the user’s library and a success message is displayed
5. The user is redirected to their library of playlists

Alternate Path: The user may choose to discard the generated playlist instead of adding it to their library

Exception Path: If the user does not have any listening history a playlist will not be generated and an error will be displayed

Pre-condition:

- User must have a valid account
- User must have listening history
- User must be logged in

Post-condition: Playlist is generated based on user listening history and the user is sent to their library of playlists

Use Case no.: 4

Use Case Name: Export playlist

Actors: User, Spotify API, External Tools Integration

Description:

1. The user navigates to the playlist library on the dashboard
2. The user selects a playlist from their library
3. The user clicks the “Export Playlist” button
4. The playlist is created on the user’s Spotify account using the Spotify API and a success message is displayed
5. The user is returned to their playlist library

Alternate Path: None

Exception Path: If the user does not have a Spotify account connected an error is displayed prompting them to connect one

Pre-condition:

- User must have a valid account
- User must be logged in
- System must be given access to the user’s Spotify account

Post-condition: Playlist is successfully created on the Spotify account connected to the user’s profile and the user is returned to their library of playlists

Use Case no.: 5

Use Case Name: Collaborate on playlist

Actors: User, Core System, Playlist Versioning Manager

Description:

1. The user navigates to a playlist they own.
2. The user enables collaborative mode.
3. The user invites another registered user to collaborate.
4. The invited user accepts the collaboration request.
5. Both users may add or remove songs from the playlist.
6. Changes are tracked and saved as playlist versions.

Alternate Path: The playlist owner may assign different permission levels (view-only or edit).

Exception Path: If the invited user does not have a valid account, an error is displayed.

Pre-condition:

- User must be logged in
- Playlist must exist
- Invited user must have a valid account

Post-condition: The playlist becomes collaborative and authorized users may edit it.

Use Case no.: 6

Use Case Name: Receive notifications

Actors: User, Core System

Description:

1. A system event occurs (playlist invite, export success, collaboration change).
2. The system generates a notification for the affected user.
3. The notification is stored in the Notifications table.
4. The user is alerted via the dashboard interface.
5. The user views and optionally marks the notification as read.

Alternate Path: The user may disable certain notification types in settings.

Exception Path: If notification storage fails, the system logs the error but continues core operations.

Pre-condition:

- User must have a valid account
- A triggering system event must occur

Post-condition: The user is informed of the event and notification status is updated.

Use Case no.: 7

Use Case Name: Compare user playlist

Actors: User, Core System, Playlist Versioning Manager, Audio Analytics System

Description:

1. The user navigates to either their own playlist or the playlist of another user
2. The user clicks the “Compare” button on the playlist page
3. The user is displayed a list of the playlists in their library and must choose which playlist to compare to
4. The playlists are compared by the Audio Analytics System
5. The user is provided a report detailing similarities and differences between the two selected playlists

Alternate Path: None

Exception Path: If the user does not have any of their own playlists to compare to an error is displayed prompting them to create a playlist in order to use the comparison function

Pre-condition:

- User must have a valid account
- User must be logged in
- User must have a valid playlist to compare to

Post-condition: A report comparing the two playlists selected by the user is displayed and when closed the user is returned to the playlist page from step 1

Use Case no.: 8

Use Case Name: Discover artists

Actors: User, Core System, Discovery and Recommendation System, User Management System

Description:

1. The user selects the “Discover Artists” section on the dashboard
2. The user inputs desired filters (genre, year, monthly listeners)
3. The user clicks “Search”
4. The user is provided a list of artists matching the selected filters

Alternate Path: The user may choose to leave the filters empty in which case their listening history will be used to provide a list of artists similar to those they already listen to

Exception Path: If no artists are found matching the users filters an error is displayed prompting the user to try different filters

Pre-condition:

- User must have a valid account
- User must be logged in

Post-condition: The user is provided with a list of artists that they may be interested in with some stats of each respective artist

Use Case no.: 9

Use Case Name: View trending songs

Actors: User, Core System

Description:

1. User navigates to the “Trending Songs” section on the dashboard
2. The page is populated with a ranked list of the top 50 currently trending songs

Alternate Path: None

Exception Path: If the trending score of any two or more songs are the same those songs will be ordered alphabetically

Pre-condition:

- User must have a valid account
- User must be logged in

Post-condition: The user is provided with a ranked list of the top 50 currently trending songs

Use Case no.: 10

Use Case Name: View user music preference analytics

Actors: User, Audio Analytics System

Description:

1. The user navigates to the “Music Preference Analytics” section.
2. The system retrieves aggregated listening history data.
3. The system displays summaries such as top genres, total playtime, and trends over selected timeframes.

4. The user may filter analytics by timeframe (week, month, year).
5. The analytics dashboard updates dynamically based on selection.

Alternate Path: If no timeframe is selected, the system defaults to the most recent week.

Exception Path: If no listening history exists, the system displays a message indicating insufficient data.

Pre-condition:

- User must have a valid account
- User must be logged in

Post-condition: The user is presented with a detailed analytics dashboard reflecting their music preferences.

5.2 Requirements

1. User Profile System- Handles login, account info, and saved user settings.
2. Description-based Recommendation- Creates playlists based only on what the user types.
3. Notification System- Sends alerts like playlist invites, export success, or system updates.
4. User Preference Recommendation- Makes a playlist based on what the user has listened to before.
5. Audio Feature Analysis- Breaks songs into basic traits like speed, key, and energy.
6. Collaborative Playlists / Friends- Lets users share playlists and edit them together.
7. User Preference Analytics- Records what the user clicks, plays, skips, or saves and summarizes what kind of music the user likes over different timeframes.
8. Popularity Recommendation- Recommends songs that are trending right now.
9. Playlist Similarity- Compares 2 playlists and shows similarity in terms of mood, genre, etc.
10. Smart Playlist versioning- Let users save and compare playlists over time.
11. Playlist Exports- Saves playlists made in the app to Spotify.
12. Artist Discovery- Find artists based on User’s profile and other filters given by the user.

Client requirements:

13. Mood Tags- Lets users tag songs with moods like "chill" or "hyped" so playlists reflect emotional vibes more accurately.
14. Quick Playlist Rename - Easily rename playlists rename without going into multiple menus
15. Light/Dark Mode Toggle: Allows users to switch between light and dark themes to improve comfort and visibility.

Requirement Descriptions

Requirement number: 01

Use Case number: 01

Introduction: Handle login, account information, and saved user settings for authenticated users

Inputs: user_id, username, email, password, theme_pref

Requirements Description:

- Must allow users to register with a unique email address
- Must securely hash and store user passwords in the database
- Must validate login credentials before granting access
- Must allow users to log out and terminate their session
- Must allow users to update account information and theme preference
- Must prevent duplicate account creation using the same email

Outputs: User account is successfully created, updated, logged in, or logged out. Appropriate success or error messages are displayed.

Requirement number: 02

Use Case number: 02

Introduction: Create a playlist based solely on a natural language description provided by the user

Inputs: user_id, user_description, optional user constraints

Requirements Description:
- Must process the user's text input using the LLM interpretation module
- Must extract mood, genre, and contextual indicators from the description
- Must retrieve songs whose audio features align with the interpreted mood
- Must exclude songs negatively rated or frequently skipped by the user
- User must be able to save or discard the generated playlist

Outputs: A playlist matching the user's description is generated and either saved to the user's library or discarded.

Requirement number: 03

Use Case number: 06

Introduction: Send notifications to users regarding relevant system events

Inputs: notification_id, user_id, notification_type, related_id

Requirements Description:
- Must generate notifications when system events occur (playlist invite, export success, updates)
- Must store notifications in the Notifications table
- Must display unread notifications on the user dashboard
- Must allow users to mark notifications as read
- Must support multiple notification types

Outputs: User receives notifications related to system activity and can view or manage them from the dashboard.

Requirement number: 04

Use Case number: 03

Introduction: Generate a playlist based on the listening history of the user

Inputs: user_id, listen_id, User Constraints

Requirements Description:

- Must use top genres and artists from listening history to populate playlist with songs
- Must exclude songs that have been negatively rated by the user
- User must be able to save or discard the playlist that has been generated

Outputs: If the user has chosen to save the playlist, it is saved in their library and the user is directed to their library. Otherwise, the playlist is not saved and the user is returned to the dashboard.

Requirement number: 06

Use Case number: 05

Introduction: Allow users to collaborate on playlists and manage shared access

Inputs: playlist_id, user_id, collaborator_user_id, permission_level

Requirements Description:

- Must allow playlist owners to enable collaborative mode
- Must allow users to invite other registered users to collaborate
- Must store collaborator permissions in the Collaborators table
- Must allow collaborators with proper permissions to add or remove songs
- Must track which user added each song to the playlist
- Must prevent unauthorized users from modifying collaborative playlists

Outputs: Collaborative playlists are shared among authorized users and edits are properly tracked.

Requirement number: 07

Use Case number: 10

Introduction: Record what the user plays, skips, saves or rates in a detailed listening history over different timeframes

Inputs: listen_id, user_id, song_id

Requirements Description:

- Log user interactions in real time

- Insert or update records in listening history database for each song played
- Update records to reflect user interactions after the user has moved to the next song or after 10 minutes without any new user interaction
- Data will be aggregated into different timeframes: last week, month, 6 months, year
- Provide dashboard to display listening history data to user
- Listening history data will be available to search and playlist generation services

Outputs: User interactions with songs are logged and recorded to be displayed in a digestible dashboard and used for other systems where listening history is pertinent

Requirement number: 08

Use Case number: 09

Introduction: Recommend songs that are trending right now

Inputs: Aggregated user listening data

Requirements Description:

- Analyze aggregated user listening data to generate a trending score for each song based on number of listens
- Generate ranked list of top 50 songs based on trending score
- List must be kept up to date by generating again every hour

Outputs: A dynamically updated list of trending songs is displayed to the user, featuring songs that currently have high engagement across the app

Requirement number: 09

Use Case number: 07

Introduction: Compare two playlists and display similarity metrics based on audio and genre data

Inputs: user_id, playlist_id_1, playlist_id_2

Requirements Description:

- Must retrieve songs and associated audio feature data for both playlists
- Must analyze similarity based on genre overlap, mood tags, and audio traits
- Must calculate a similarity score between the two playlists
- Must generate a comparison report outlining similarities and differences
- Must return the user to the original playlist page after the report is closed

Outputs: A comparison report is displayed showing similarity percentage, shared genres, mood alignment, and differences between playlists.

Requirement number: 11

Use Case number: 4

Introduction: Export playlists made or generated in the app to external music platforms

Inputs: user_id, playlist_id

Requirements Description:

- Must support exporting playlists to Spotify
- Must verify that the user has authorized the app to access and create playlists on the user's external account
- Must find songs on external platform using respective song meta data
- Notify user after playlist export if any song could not be found

Outputs: The selected playlist is successfully created on the external platform, and a confirmation message is displayed with any songs that could not be found

Requirement number: 12

Use Case number: 08

Introduction: Find artists based on User's profile and other filters given by the user.

Inputs: user_id, listen_id, User given parameters

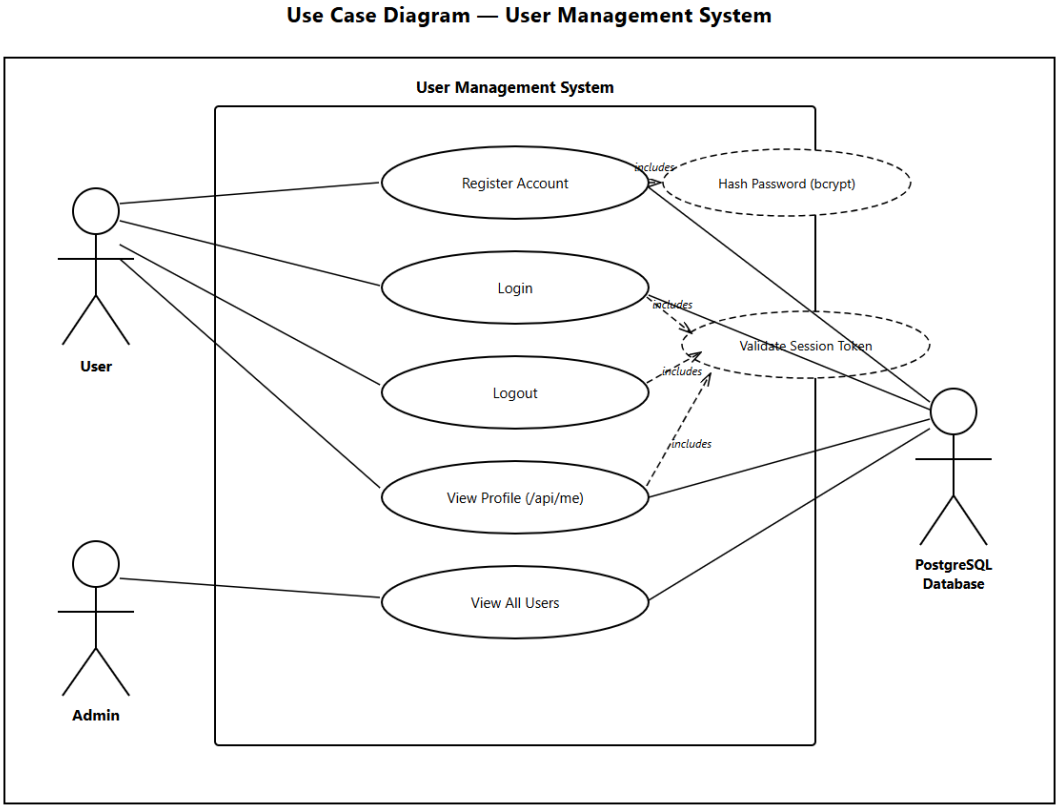
Requirements Description:

- Must allow users to filter based on genre, year, and number of listeners per month
- Must be able to send the user to the artist page by clicking on a specific artist in the provided results
- User must be able to sort the provided results by artist name and monthly listeners
- If no filters are applied the system must prioritize top artists within genres the user has listened to within the past week
- Each entry must display name, picture, genre, and monthly listeners of the artist
- List must be sorted by monthly listeners by default

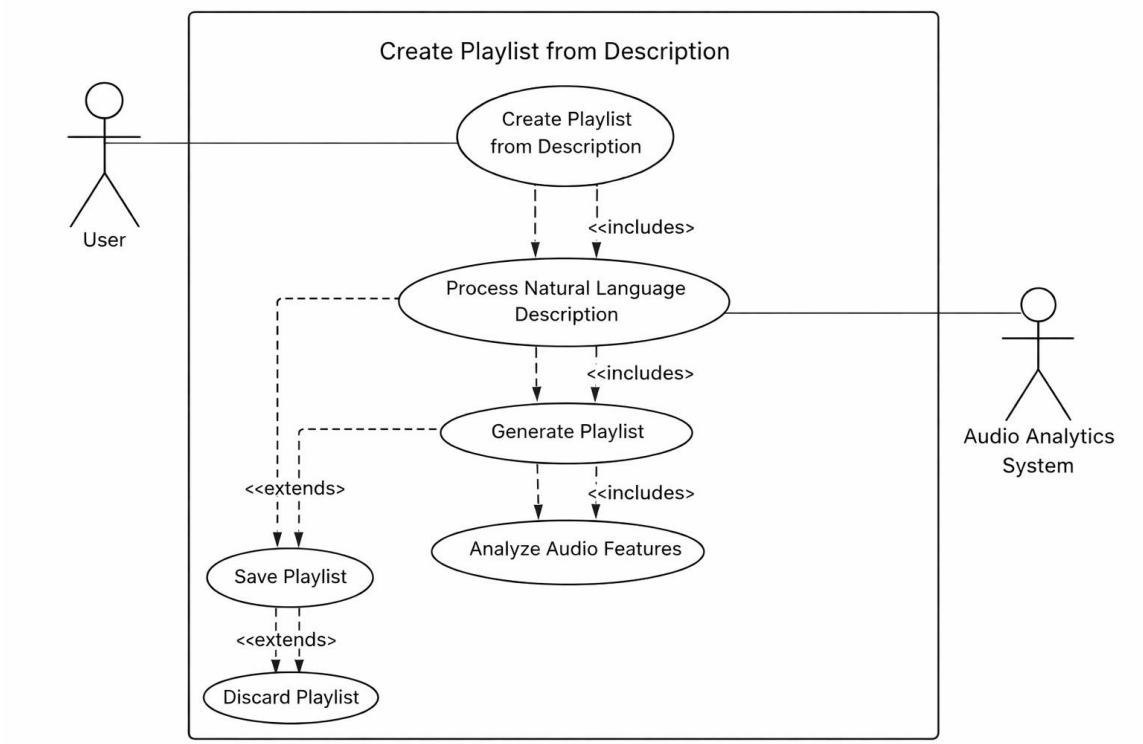
Outputs: Lists artists matching the user provided filters by popularity, or uses the listening history of the user if no filters are provided.

5.3 Use Case Diagrams

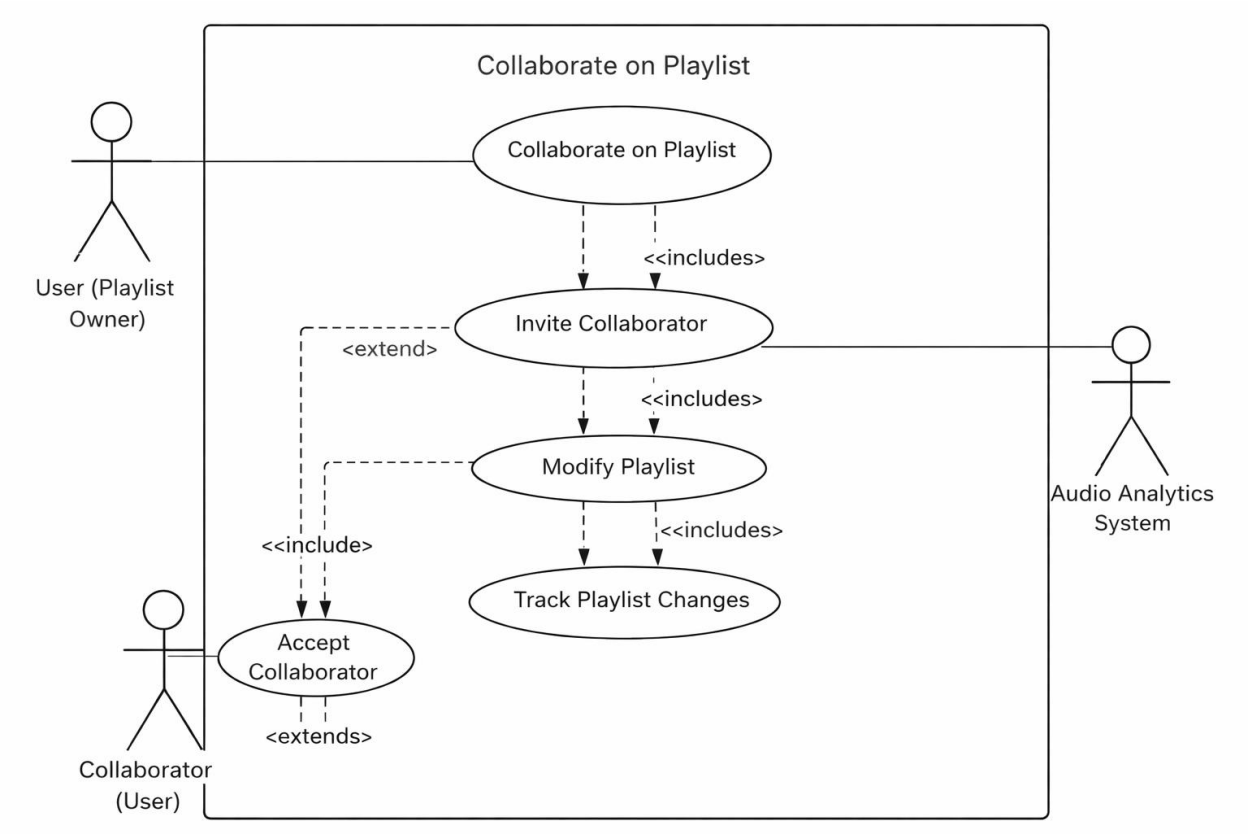
Use Case 1:



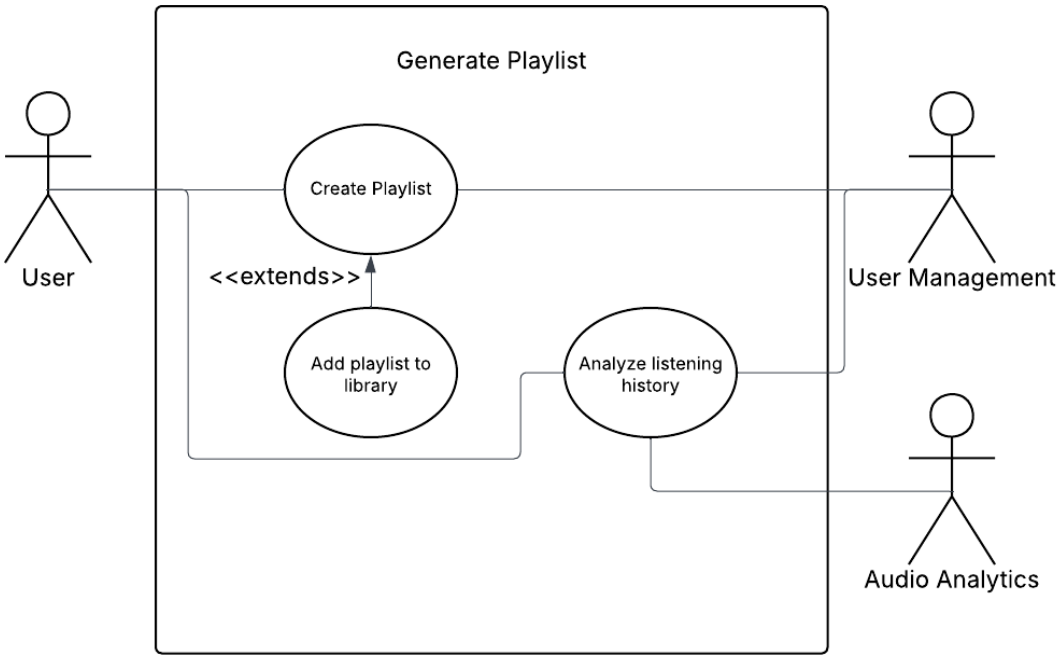
Use Case 2:



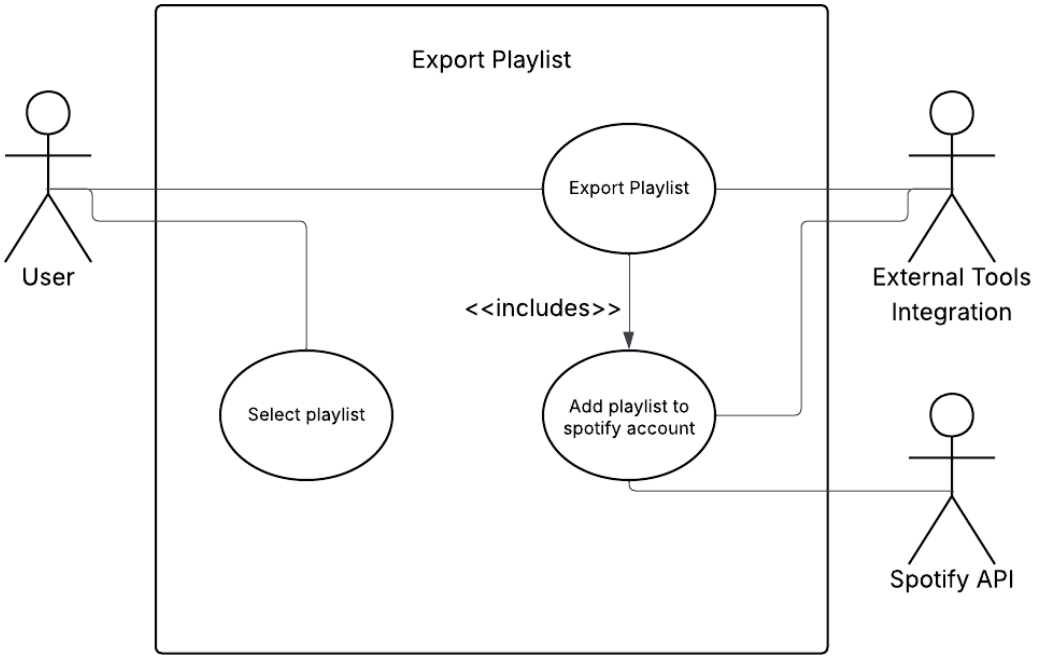
Use Case 5



Use Case 3



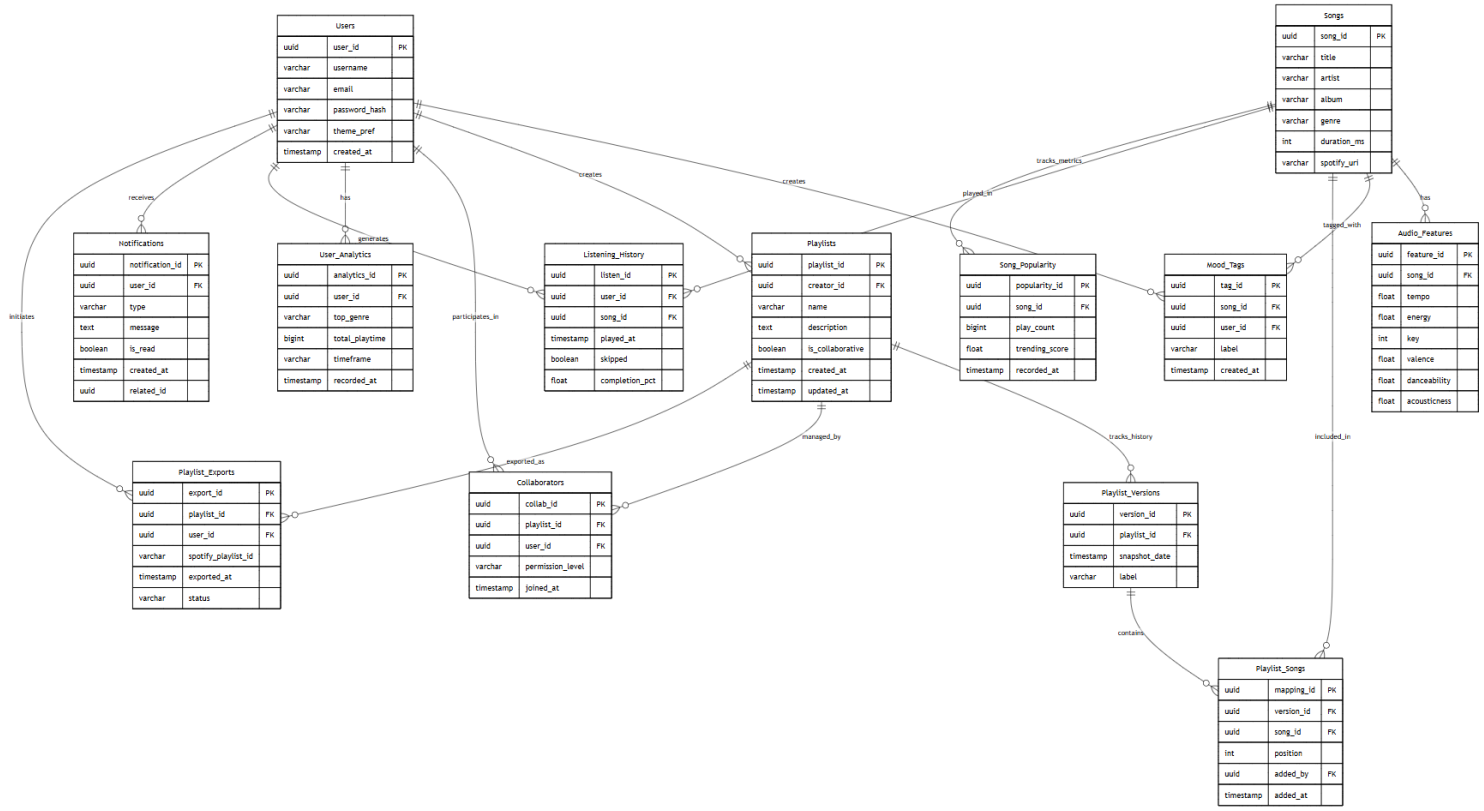
Use Case 4



6.1 Database Management

We chose to use PostgreSQL for this project because it efficiently handles multi-table joins, which we're going to have a lot of in this project. Its JSONB support and indexing allows for faster deep analysis of semi-structured audio features and user generated tags. It also correctly handles concurrent transactions, which prevents data corruption when multiple users collaborate on the same playlist simultaneously.

ER Diagram:



6.2 System Database Tables & Relationships

Users

- user_id: UUID (PK)
- username: VARCHAR(255)
- email: VARCHAR(255)
- password_hash: VARCHAR(255)
- theme_pref: VARCHAR(50)
- created_at: TIMESTAMP

Songs

- song_id: UUID (PK)
- title: VARCHAR(255)
- artist: VARCHAR(255)
- album: VARCHAR(255)
- genre: VARCHAR(100)
- duration_ms: INTEGER

- spotify_uri: VARCHAR(255)

Audio_Features

- feature_id: UUID (PK)
- song_id: UUID (FK) -> Songs.song_id
- tempo: FLOAT
- energy: FLOAT
- key: INTEGER
- valence: FLOAT
- danceability: FLOAT
- acousticness: FLOAT

Mood_Tags

- tag_id: UUID (PK)
- song_id: UUID (FK) -> Songs.song_id
- user_id: UUID (FK) -> Users.user_id
- label: VARCHAR(50)
- created_at: TIMESTAMP

Playlists

- playlist_id: UUID (PK)
- creator_id: UUID (FK) -> Users.user_id
- name: VARCHAR(255)
- description: TEXT
- is_collaborative: BOOLEAN
- created_at: TIMESTAMP
- updated_at: TIMESTAMP

Playlist_Versions

- version_id: UUID (PK)
- playlist_id: UUID (FK) -> Playlists.playlist_id
- snapshot_date: TIMESTAMP
- label: VARCHAR(100)

Playlist_Songs

- mapping_id: UUID (PK)
- version_id: UUID (FK) -> Playlist_Versions.version_id
- song_id: UUID (FK) -> Songs.song_id
- position: INTEGER
- added_by: UUID (FK) -> Users.user_id
- added_at: TIMESTAMP

Collaborators

- collab_id: UUID (PK)
- playlist_id: UUID (FK) -> Playlists.playlist_id
- user_id: UUID (FK) -> Users.user_id
- permission_level: VARCHAR(50)
- joined_at: TIMESTAMP

Listening_History

- listen_id: UUID (PK)
- user_id: UUID (FK) -> Users.user_id
- song_id: UUID (FK) -> Songs.song_id
- played_at: TIMESTAMP
- skipped: BOOLEAN
- completion_pct: FLOAT

User_Analytics

- analytics_id: UUID (PK)
- user_id: UUID (FK) -> Users.user_id
- top_genre: VARCHAR(100)
- total_playtime: BIGINT
- timeframe: VARCHAR(50)
- recorded_at: TIMESTAMP

Notifications

- notification_id: UUID (PK)
- user_id: UUID (FK) -> Users.user_id

- type: VARCHAR(100)
- message: TEXT
- is_read: BOOLEAN
- created_at: TIMESTAMP
- related_id: UUID

Song_Popularity
- popularity_id: UUID (PK)
- song_id: UUID (FK) -> Songs.song_id
- play_count: BIGINT
- trending_score: FLOAT
- recorded_at: TIMESTAMP

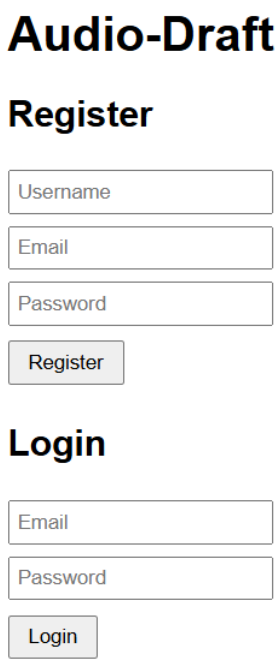
Playlist_Exports
- export_id: UUID (PK)
- playlist_id: UUID (FK) -> Playlists.playlist_id
- user_id: UUID (FK) -> Users.user_id
- spotify_playlist_id: VARCHAR(255)
- exported_at: TIMESTAMP
- status: VARCHAR(50)

7.0 Implementation & Screenshots

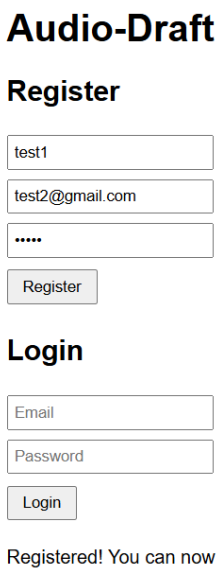
Pictures and Links

Github Link: https://github.com/Srithwak/Audio-Draft

Login Page:



Registration Success:



Duplicate Registration:

Audio-Draft

Register

test1

test2@gmail.com

•••••

Register

Login

Email

Password

Login

duplicate key value violates unique constraint "users_email_key" DETAIL: Key (email)= (test2@gmail.com) already exists.

Landing Page after login:

Audio-Draft

Logged in as: test1

Logout

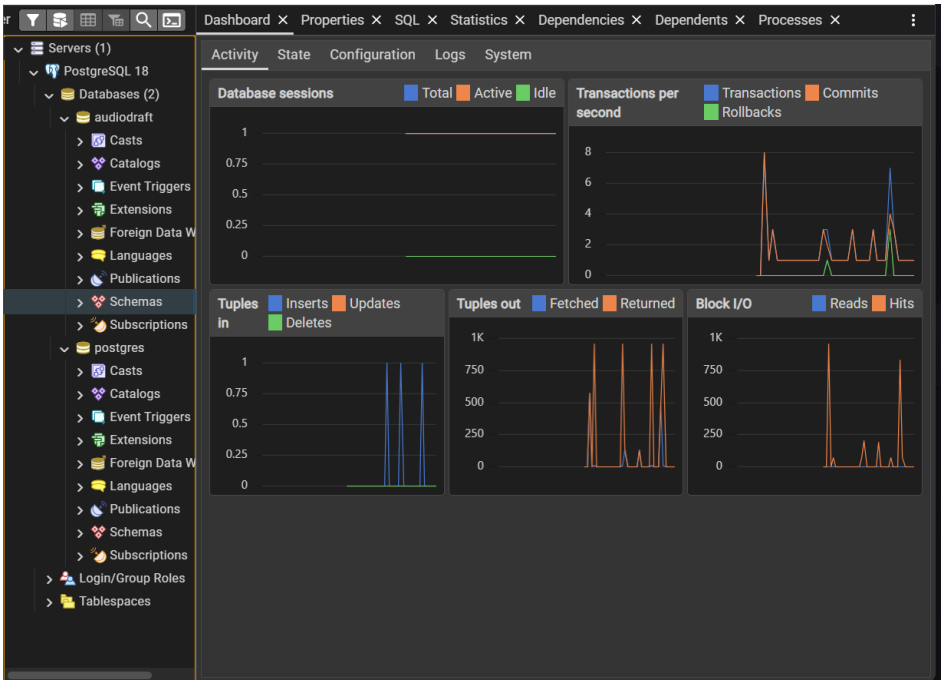
Songs

No songs in database.

PostgreSQL calls:

```
Debugger PIN: 258-231-063
127.0.0.1 - - [19/Feb/2026 21:48:53] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2026 21:48:53] "GET /style.css HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2026 21:48:53] "GET /app.js HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2026 21:48:53] "GET /api/me HTTP/1.1" 401 -
127.0.0.1 - - [19/Feb/2026 21:49:04] "POST /api/register HTTP/1.1" 201 -
127.0.0.1 - - [19/Feb/2026 21:49:11] "POST /api/login HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2026 21:49:11] "GET /api/songs HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2026 21:50:01] "POST /api/logout HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2026 21:50:03] "GET / HTTP/1.1" 304 -
127.0.0.1 - - [19/Feb/2026 21:50:03] "GET /style.css HTTP/1.1" 304 -
127.0.0.1 - - [19/Feb/2026 21:50:03] "GET /app.js HTTP/1.1" 304 -
127.0.0.1 - - [19/Feb/2026 21:50:03] "GET /api/me HTTP/1.1" 401 -
127.0.0.1 - - [19/Feb/2026 21:50:44] "POST /api/register HTTP/1.1" 400 -
127.0.0.1 - - [19/Feb/2026 21:51:10] "POST /api/register HTTP/1.1" 201 -
127.0.0.1 - - [19/Feb/2026 21:52:02] "POST /api/register HTTP/1.1" 400 -
127.0.0.1 - - [19/Feb/2026 21:52:27] "POST /api/login HTTP/1.1" 200 -
127.0.0.1 - - [19/Feb/2026 21:52:27] "GET /api/songs HTTP/1.1" 200 -
```

Pgadmin4 Logs:



GitHub Board:

Srithwak Implemented Desktop App 454710c · 4 days ago 3 Commits

backend	Database and build script setup	4 days ago
database	Database and build script setup	4 days ago
frontend	Implemented Desktop App	4 days ago
.gitignore	Database and build script setup	4 days ago
README.md	Implemented Desktop App	4 days ago
build.bat	Implemented Desktop App	4 days ago
desktop_app.py	Implemented Desktop App	4 days ago
documentation.md	Implemented Desktop App	4 days ago
init_db.py	Database and build script setup	4 days ago
requirements.txt	Implemented Desktop App	4 days ago

README

Audio-Draft

A Music Intelligence Platform built with Python, Flask, PostgreSQL, and flaskwebgui.

Runs as a native desktop application.

Quick Start

1. Install Python 3.10+ and PostgreSQL.
2. Clone this repository.
3. Run `build.bat` and enter your PostgreSQL password when prompted.
4. A desktop window will open with the application.

See `documentation.md` for full details.

How to compile and run:

1. Install Python 3.10+, git, pgadmin4 (Setup for postgresql 18 and enter password)

2. Open Terminal in designated folder and enter- git clone https://github.com/Srithwak/Audio-Draft.git

3. Enter cd Audio-Draft, and then ./build.bat. Enter your pgadmin4 password when prompted.

The script will automatically create necessary libraries, set up your database, and launch the application.

4. Once the app opens, you can register and login to the database.